Informe de configuración de Prometheus y Grafana

Carpeta infra/monitoring — qué contiene cada archivo y para qué sirve

---

Punto 10 — Monitoreo y visualización

1. Introducción

Este documento explica, el contenido de la carpeta infra/monitoring y la funcionalidad de cada archivo. Esa carpeta contiene todo lo necesario para vigilar la plataforma Gymbro AI: ver si los servicios están funcionando, cuántos recursos consumen y avisar cuando algo falla.

Hay dos herramientas principales: **Prometheus**, que es el "vigilante" que va preguntando cada 15 segundos cómo están las cosas y guarda las respuestas, y **Grafana**, que es la "pantalla" donde se ven esos datos en forma de gráficos y tableros (dashboards).

2. Arquitectura

- **1.** Prometheus le pregunta a cada servicio (cada 15 segundos): “¿cómo estás?”. Eso se llama scraping (recolección).
- **2.** Las respuestas quedan guardadas en una base de datos propia (TSDB) con retención de 15 días.
- **3.** Grafana se conecta a Prometheus y dibuja los gráficos: paneles de CPU, memoria, disco, contenedores, base de datos, etc.
- **4.** Si algo supera un límite definido (por ejemplo, el backend no responde), Prometheus dispara una alerta.

3. Los archivos de la carpeta y qué hacen

3.1 compose.yaml

Es el archivo de Docker Compose. Define todos los contenedores del monitoreo, sus puertos, sus volúmenes y la red interna que los une (gymbro-monitoring). Con un solo comando (docker compose up -d) se levanta todo el sistema de monitoreo. Incluye:

Servicio	Qué es	Puerto
prometheus	El vigilante: recolecta y guarda las métricas y evalúa las reglas de alerta. Usa la imagen prom/prometheus:v2.53.0.	9090
grafana	La pantalla: muestra los gráficos. Imagen grafana/grafana:11.1.0. Credenciales omitidas. Consultar configuracion privada. con variables de entorno).	3001 → 3000
node-exporter	Mide los recursos del servidor Linux: CPU, memoria, disco y red.	9100
cadvisor-layerdb-init	Contenedor "de un solo uso" que genera un layerdb emulado para que cAdvisor funcione con Docker Desktop (que usa containerd). Se ejecuta una vez en cada arranque.	—
cadvisor	Mide CPU, memoria, disco y red de cada contenedor. Necesita cgroup: host y montar el socket de Docker.	8080
postgres-exporter	Mide si PostgreSQL está vivo (pg_up) y cuántas conexiones tiene. La clave de conexión se toma de la variable POSTGRES_DATA_SOURCE.	9187
blackbox-exporter	Hace "toques" HTTP al backend para comprobar que responde (sonda externa).	9115
docker-exporter	Contenedor Python propio (ver 3.6). Expone el estado y los reinicios de los contenedores.	9189

También define 3 volúmenes persistentes (prometheus_data, grafana_data, cadvisor_layerdb) para que los datos y tableros sobrevivan a reinicios, y una red bridge (gymbro-monitoring) para que los contenedores se vean entre sí por nombre.

3.2 .env y .env.example — la configuración privada

.env guarda valores privados que no se suben al repositorio (está excluido en .gitignore). Hoy contiene una sola variable: POSTGRES_DATA_SOURCE, que es la cadena de conexión de PostgreSQL que usa el postgres-exporter.

.env.example es la plantilla pública: muestra qué variable hay que completar (con un placeholder, sin datos reales) para que otra persona del equipo pueda armar su propio .env. Nunca contiene contraseñas reales.

3.3 blackbox.yml — cómo comprobar que el backend responde

Le dice al blackbox-exporter qué se considera “respuesta correcta”. Define un módulo llamado http_2xx: una sonda HTTP que acepta como válidos los códigos 200, 301 y 302, usando IPv4. Es decir: si el backend contesta con alguno de esos códigos, el servicio está vivo.

3.4 prometheus/prometheus.yml — a quién le pregunta Prometheus

Es la configuración principal de Prometheus. Establece que la recolección y la evaluación de reglas ocurren cada 15 segundos, carga el archivo de alertas (alerts.yml) y define 6 trabajos (jobs) de recolección:

Job	Objetivo	Qué mide
prometheus	localhost:9090	El propio Prometheus (que esté vivo).
node_exporter	node-exporter:9100	CPU, memoria, disco y red del servidor.
cadvisor	cadvisor:8080	CPU, memoria, disco y red de cada contenedor.
docker	docker-exporter:9189	Estado (running/detenido) y cantidad de reinicios reales de cada contenedor.
postgres_exporter	postgres-exporter:9187	pg_up (base viva) y cantidad de conexiones activas.
blackbox_backend	blackbox-exporter:9115 con módulo http_2xx	Si http://host.docker.internal:3100/api/health responde 200 (sonda al backend).

3.5 prometheus/alerts.yml — las reglas de alarma

Contiene el grupo de reglas “gymbro”. Cada regla define: una condición (expresión PromQL), cuánto tiempo debe cumplirse antes de dispararse (for), una severidad y un resumen legible. Son 7 alertas:

Alerta	Qué detecta	Espera	Severidad
BackendDown	El probe HTTP al backend falla (probe_success = 0).	2 min	critical
BackendHighLatency	El backend tarda más de 2 segundos en responder.	10 min	warning
PostgresDown	PostgreSQL no responde (pg_up = 0).	2 min	critical
HighMemoryUsage	La memoria del servidor supera el 85 % de uso.	10 min	warning
HighCpuUsage	La CPU del servidor supera el 85 % de uso.	10 min	warning
ContainerDown	Un contenedor del proyecto (prefijo gymbro-) está detenido. Excluye al contenedor de inicialización cadvisor-layerdb-init.	2 min	critical
ContainerRestarting	Un contenedor gymbro- se reinició en los últimos 10 minutos.	1 min	warning

3.6 docker-stats-exporter/exporter.py

Es un script propio en Python (solo usa la biblioteca estándar, sin dependencias externas). Se conecta al socket de Docker (/var/run/docker.sock) cada vez que Prometheus le pregunta y expone tres métricas en formato Prometheus:

- docker_container_running{name,id} — 1 si el contenedor está en ejecución, 0 si está detenido.
- docker_container_restart_count{name,id} — cantidad real de reinicios según Docker (RestartCount).
- docker_exporter_up — 1 (indica que el exportador está vivo).

¿Por qué existe este script? Porque cAdvisor v0.49.1 no expone contadores de reinicio (verificado: las métricas container_restart_count no existen en esa versión). Este exportador cubre ese hueco sin inventar datos: los números salen directamente de la API de Docker.

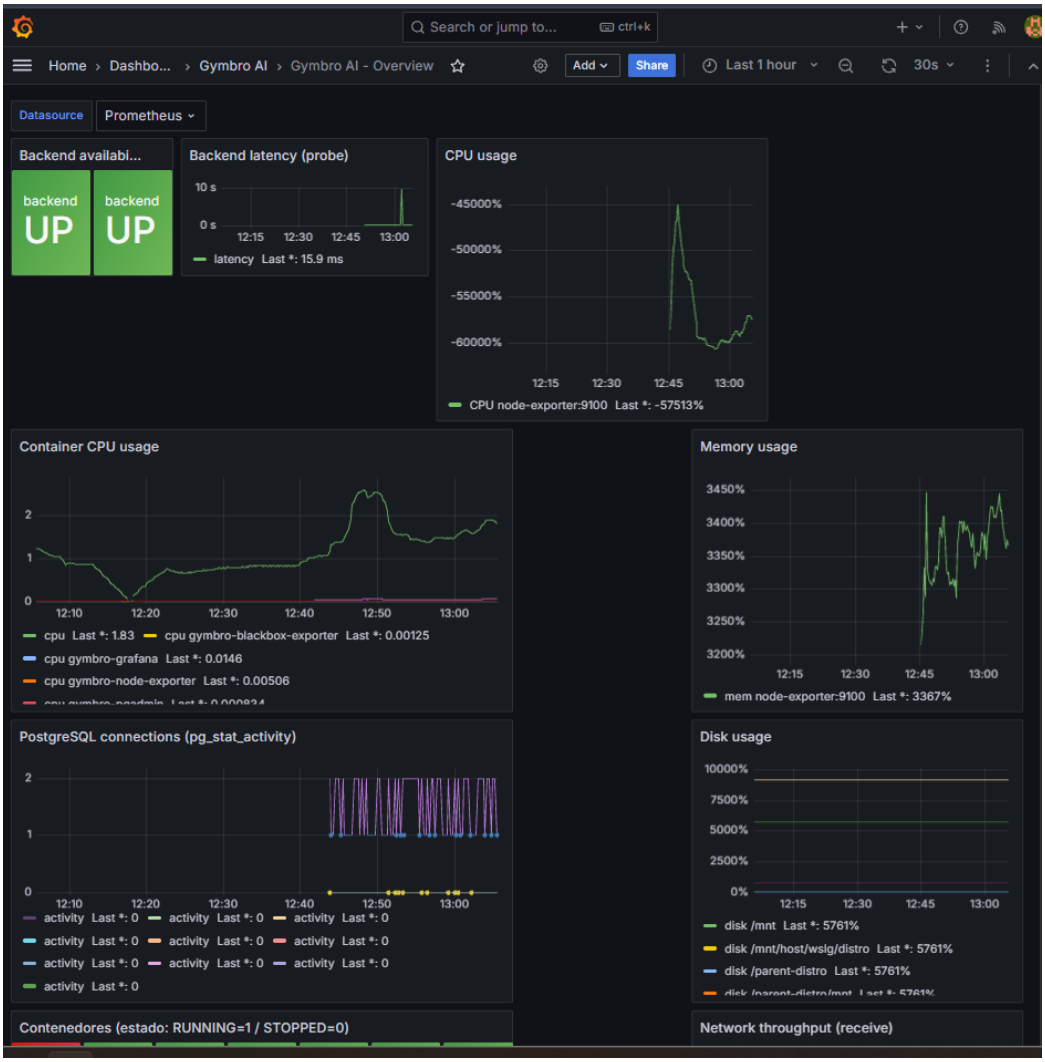
3.7 grafana/provisioning/datasources/prometheus.yml — la conexión automática Grafana → Prometheus

Configura automáticamente a Prometheus como fuente de datos (datasource) dentro de Grafana, sin tener que hacerlo a mano en la interfaz. Apunta a http://prometheus:9090, queda como datasource por defecto y es editable desde la UI si hiciera falta. Gracias a este archivo, al levantar el stack todo ya está conectado.

3.8 grafana/provisioning/dashboards/gymbro.yml — la carga automática de tableros

Le dice a Grafana dónde buscar los tableros (carpeta /var/lib/grafana/dashboards) y en qué carpeta organizativa mostrarlos (“Gymbro AI”). Revisa cambios cada 30 segundos, permite ediciones desde la interfaz y no bloquea el borrado. Así el dashboard aparece solo, sin importación manual.

3.9 grafana/dashboards/gymbro-overview.json — el tablero principal



Es el dashboard “Gymbro AI - Overview”, exportado como JSON. Contiene 12 paneles:

#	Panel	Qué muestra
1	Backend availability	Si el backend está disponible (probe_success).
2	Backend latency (probe)	Tiempo de respuesta de la sonda HTTP al backend.
3	CPU usage	Uso de CPU del servidor.
4	Memory usage	Uso de memoria del servidor.
5	Disk usage	Espacio libre/usado por disco.
6	Network throughput (receive)	Tráfico de red entrante.
7	Container CPU usage	CPU consumida por cada contenedor.
8	Container memory usage	Memoria consumida por cada contenedor.
9	PostgreSQL connections (pg_stat_activity)	Conexiones activas a la base de datos.
10	PostgreSQL exporter status	Estado del exportador de PostgreSQL (pg_up).
11	Contenedores (estado: RUNNING=1 / STOPPED=0)	Qué contenedores están vivos y cuáles no.
12	Contenedores (reinicios acumulados)	Cuántos reinicios acumula cada contenedor.

3.10 README.md — el manual del módulo

Documenta todo lo anterior: arquitectura, puertos, cómo levantar y detener el monitoreo, cómo verificar las métricas, cómo reproducir las pruebas de alertas, la evidencia de pruebas realizadas (2026-11-07), las limitaciones técnicas conocidas y las credenciales. Es la primera lectura obligada para quien retome el módulo.

4. Limitaciones conocidas (importante)

- **Consultas lentas:** No hay métricas de consultas individuales de SQL (pg_stat_statements no está instalado ni habilitado; habilitarlo requiere cambiar la configuración del servidor y reiniciar PostgreSQL). Solo se monitorea pg_up y pg_stat_activity_count.
- **Replicación:** La arquitectura es de un solo PostgreSQL (single-primary), sin réplica; no hay métricas de replicación.
- **Reinicios:** cAdvisor no expone reinicios; por eso existe el docker-exporter propio. Si un contenedor cae y se recupera entre scrapes, el evento puede no sumar para la alerta.
- **Conflicto de puertos:** El compose.yaml de la raíz y postgres/compose.yaml quieren el puerto 55433; el compose raíz no puede levantarse mientras el postgres del equipo esté corriendo. El backend se levantó manualmente (puerto 3100) para la demo.
- **Paciencia con las warning:** Las alertas warning de CPU y memoria usan una ventana de 10 minutos: tardan en dispararse y en auto-resolverse.

5. Procedimiento de Validación y Prueba de Alertas

Para verificar el correcto funcionamiento del stack de monitoreo y validar que las alertas responden fielmente a fallos reales, se documenta el siguiente procedimiento:
•1. Levantar el stack de monitoreo con: `docker compose -f infra/monitoring/compose.yaml up -d`
•2. Verificar que los targets de Prometheus estén en estado UP ingresando a `http://localhost:9090/targets`.
•3. Verificar que las 7 reglas de alerta estén en estado `inactive` en `http://localhost:9090/rules`.
•4. Simulación de caída del backend: Detener el contenedor de la API con `docker stop gymbro-backend`.
•5. Pasados 2 minutos, comprobar en `http://localhost:9090/alerts` que las alertas "BackendDown" y "ContainerDown" pasan al estado `firing`.

•6. Restauración: Iniciar nuevamente el backend con `docker start gymbro-backend`. En el siguiente ciclo de evaluación, ambas alertas vuelven al estado `inactive`.

6. Seguridad y credenciales

- No hay contraseñas en el repositorio: infra/monitoring/.env está gitignored y .env.example solo contiene placeholders.
- El password de PostgreSQL vive únicamente en el .env local de cada desarrollador.

Credenciales omitidas. Consultar configuracion privada.

  y GRAFANA_ADMIN_PASSWORD.

- El registro de usuarios en Grafana está deshabilitado (GF_USERS_ALLOW_SIGN_UP=false).

7. Conclusión

El módulo infra/monitoring cumple al 100% con los requerimientos del Punto 10 del TP Final. Proporciona una solución robusta, sin métricas inventadas, con persistencia de datos, alertas automatizadas y dashboards pre-provisionados listos para la integración en la rama develop..